Assignment No. 1

Title

Set up a Git repository for a data science project and push it to GitHub to manage version control, track changes, and ensure code stability.

Theory

Version control is a system that records changes to files over time, allowing users to recall specific versions later. Git is a distributed version control system that enables multiple developers to work on projects simultaneously while maintaining a complete history of changes. GitHub is a cloud-based hosting service that manages Git repositories, providing collaboration features, remote backup, and project management tools.

In data science projects, version control is crucial because workflows involve continuous experimentation with datasets, preprocessing techniques, models, and parameters. Without version control, tracking changes and reproducing results becomes difficult. Git allows data scientists to maintain a structured history of experiments and collaborate efficiently.

Git follows a distributed model where every developer has a complete copy of the repository, enabling offline work and redundancy. The Git workflow consists of the working directory, staging area, and repository. GitHub extends Git by providing remote hosting, pull requests, issue tracking, and CI/CD integration.

Purpose

1. To understand the fundamentals of version control
2. To initialize and configure a Git repository for a data science project
3. To learn basic Git commands such as add, commit, push, and pull
4. To connect a local repository with GitHub
5. To track changes and maintain code stability
6. To apply best practices for data science project organization
7. To understand the role of the .gitignore file
8. To support collaborative data science development

Algorithms

Algorithm 1: Initialize Local Git Repository

Input: Project directory
Output: Initialized Git repository

Steps:

9. Navigate to project directory
10. Execute git init
11. Create project folders
12. Create README.md and .gitignore
13. Stage files using git add .
14. Commit using git commit -m "Initial commit"

Algorithm 2: Connect Local Repository to GitHub

Input: Local Git repository
Output: Remote GitHub repository linked

Steps:

15. Create a new repository on GitHub
16. Copy repository URL
17. Add remote using git remote add origin
18. Rename branch to main
19. Push code using git push -u origin main

Algorithm 3: Basic Git Workflow

Input: Modified files
Output: Changes committed and pushed

Steps:

20. Modify files
21. Check status using git status
22. Stage changes
23. Commit changes
24. Push to GitHub

Implementation Steps

Step 1: Install and Configure Git

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Step 2: Create Project Structure

```
mkdir my_data_science_project
cd my_data_science_project
mkdir data notebooks src models outputs docs
```

Step 3: Initialize Git Repository

```
git init
```

Step 4: Create Essential Files

```
echo "# My Data Science Project" > README.md
touch .gitignore
```

Step 5: Initial Commit

```
git add .
git commit -m "Initial commit: project structure"
```

Step 6: Create GitHub Repository

Create a new repository on GitHub (do not initialize with README).

Step 7: Push to GitHub

```
git remote add origin https://github.com/username/repository.git
git branch -M main
```

```
git push -u origin main
```

Step 8: Verify Repository

```
git remote -v
```

Step 9: Practice Workflow

Create analysis.py, commit, and push changes.

Step 10: View History

```
git log --oneline
git status
```

Application

- Machine learning experiment tracking
- Collaborative data analysis
- Reproducible research
- Model version tracking
- Data pipeline management

Conclusion

This assignment demonstrated how Git and GitHub support structured version control for data science projects. The exercise highlighted the importance of proper project organization, meaningful commits, and remote backups for collaborative and reproducible development.